EXPT NO:2	CONTRAST ADJUSTMENT
DATE: 12.12.25	

AIM:

Load a low-contrast grayscale image using OpenCV and enhance its visibility by applying three contrast adjustment methods:

• Linear contrast stretching using cv2.normalize()

• Histogram equalization using cv2.equalizeHist()

• CLAHE (Contrast Limited Adaptive Histogram Equalization).

Display all four images (original + three enhanced) in a grid, plot their histograms, and identify which method produces the best visual quality.

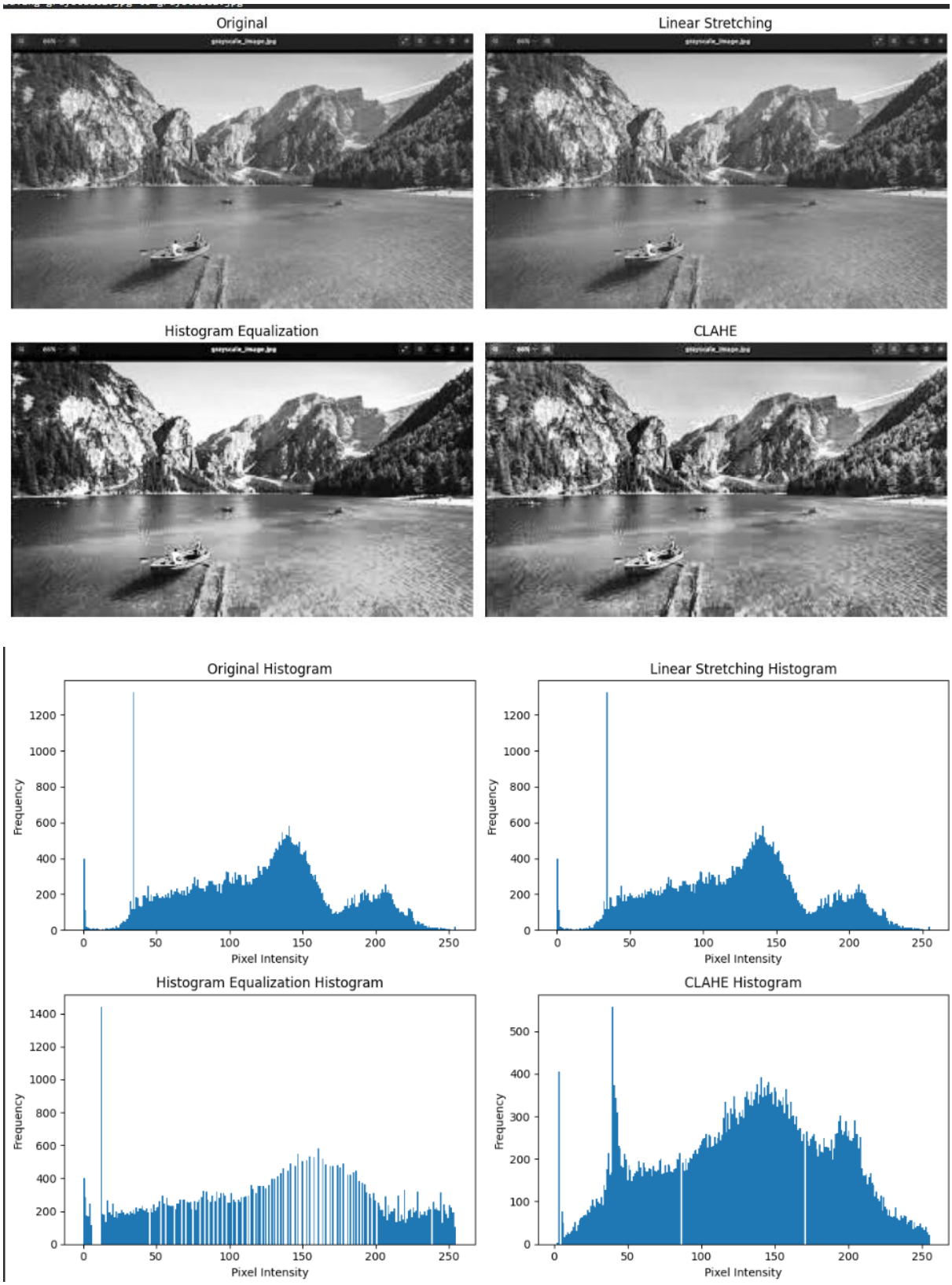
.**PROGRAM CODE:**

A) **FOR GRAY SCALE IMAGE**

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
uploaded = files.upload()
image_path = list(uploaded.keys())[0]
img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
linear = cv2.normalize(img, None, 0, 255, cv2.NORM_MINMAX)
hist_eq = cv2.equalizeHist(img)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
clahe_img = clahe.apply(img)
titles = ['Original', 'Linear Stretching', 'Histogram Equalization', 'CLAHE']
images = [img, linear, hist_eq, clahe_img]
plt.figure(figsize=(12, 8))
for i in range(4):
    plt.subplot(2, 2, i+1)
    plt.imshow(images[i], cmap='gray')
    plt.title(titles[i])
    plt.axis('off')
plt.tight_layout()
plt.show()
plt.figure(figsize=(12, 8))
for i in range(4):
    plt.subplot(2, 2, i+1)
    plt.hist(images[i].ravel(), 256)
    plt.title(titles[i] + " Histogram")
    plt.xlabel("Pixel Intensity")
    plt.ylabel("Frequency")
```

```
plt.tight_layout()
plt.show()
```

OUTPUT:



B) FOR COLOR IMAGE

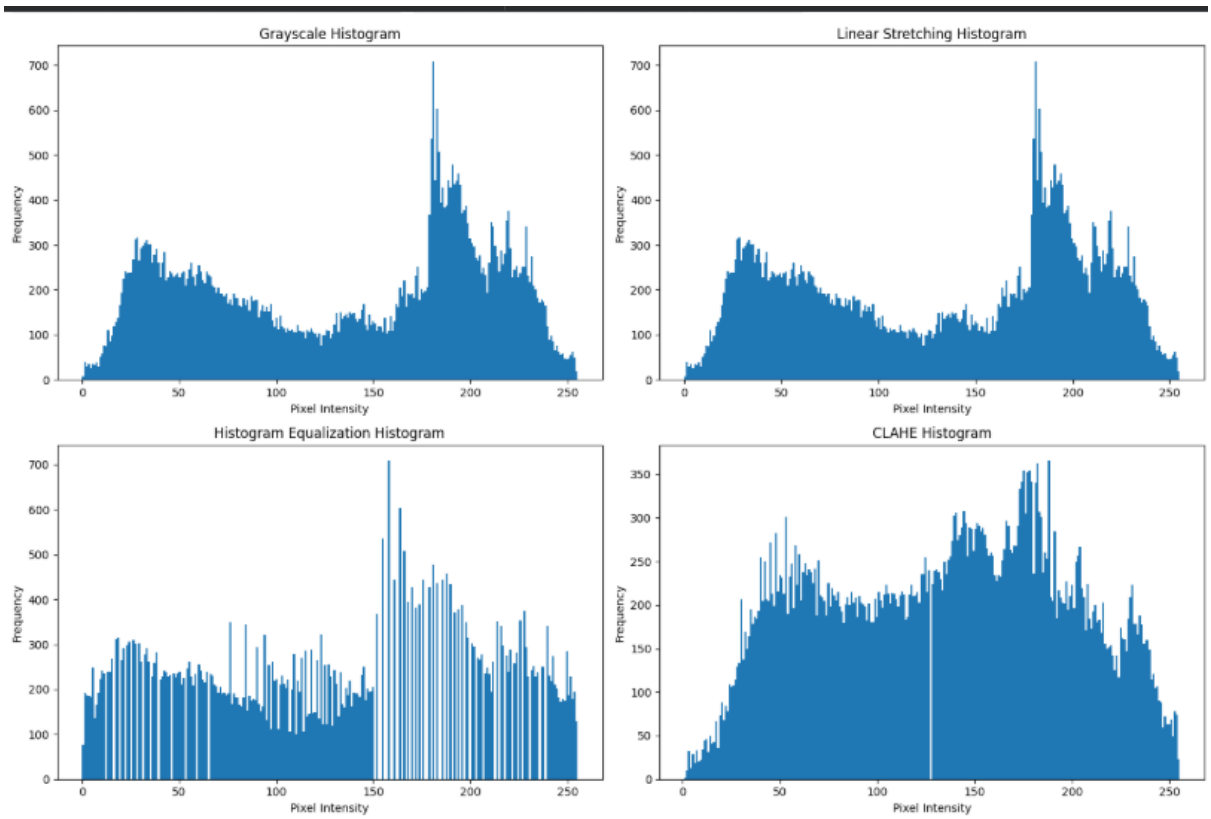
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
uploaded = files.upload()
image_path = list(uploaded.keys())[0]
color_img = cv2.imread(image_path)
color_img_rgb = cv2.cvtColor(color_img, cv2.COLOR_BGR2RGB)
img = cv2.cvtColor(color_img, cv2.COLOR_BGR2GRAY)
linear = cv2.normalize(img, None, 0, 255, cv2.NORM_MINMAX)
hist_eq = cv2.equalizeHist(img)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
clahe_img = clahe.apply(img)
titles = ['Original Color Image', 'Grayscale Image', 'Linear Stretching', 'Histogram
Equalization', 'CLAHE']
images = [color_img_rgb, img, linear, hist_eq, clahe_img]
plt.figure(figsize=(15, 10))
for i in range(len(images)):
    plt.subplot(2, 3, i+1)
    if i == 0:
        plt.imshow(images[i])
    else:
        plt.imshow(images[i], cmap='gray')
    plt.title(titles[i])
    plt.axis('off')
plt.tight_layout()
plt.show()

plt.figure(figsize=(15, 10))
hist_titles = ['Grayscale', 'Linear Stretching', 'Histogram Equalization', 'CLAHE']
hist_imgs = [img, linear, hist_eq, clahe_img]

for i in range(4):
    plt.subplot(2, 2, i+1)
    plt.hist(hist_imgs[i].ravel(), bins=256)
    plt.title(hist_titles[i] + " Histogram")
    plt.xlabel("Pixel Intensity")
    plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```

OUTPUT:



RESULT:

Thus , the program has been executed and the output is verified successfully.